Sistema de alquiler de vehículos: informe final y guion de la defensa

Proyecto final integrador · Lenguajes 2 · Ingeniería en Informática · Grupo 25

Salazar, Alejandro Nicolás (DNI 44754944)

Soto Llatser, José Ignacio (DNI 44818468)

Perez, Fernando F. Agustin (DNI 44174981)

Juarez Ruiz, David Maximiliano (DNI 44696409)

Chaparo, Samuel David (DNI 37720868)

Montoya, Tomás Benjamín (DNI 47607133)

Universidad Católica de Salta · Facultad de Ingeniería · Ingeniería en Informática

Docente: Lic. Rosa Eufracia Macaione

Fecha de entrega: 11 de noviembre de 2026.

Resumen

Este informe presenta un sistema de escritorio en Java para una agencia de alquiler de vehículos, desarrollado como proyecto final de la cátedra Lenguajes 2. El sistema tiene dos interfaces gráficas en Swing que trabajan al mismo tiempo sobre los mismos datos: una para el operario del mostrador, que administra la flota, los clientes y los contratos, y un kiosco de autoservicio en el que el cliente elige un vehículo disponible, se identifica con su DNI y recibe un ticket en PDF. El diseño separa la interfaz, los servicios con las reglas del negocio, el modelo, la persistencia y las excepciones, y aplica herencia, polimorfismo, interfaces (patrón Strategy), composición, excepciones propias y serialización. El desarrollo se organizó en 21 historias de usuario, de las cuales se completaron 18, y se programó con tests primero: el proyecto tiene 245 pruebas automáticas con JUnit 5 y una prueba de la interfaz que recorre la demostración completa. Se describen también el reparto de la defensa entre los seis integrantes y el guion de la demostración en vivo.

Palabras clave: programación orientada a objetos, Java, Swing, persistencia, historias de usuario, pruebas automatizadas

Introducción

Una agencia de alquiler necesita saber qué vehículos tiene disponibles, quién puede manejarlos y cuánto cobrar por cada alquiler. Cuando esa información vive en papel o en planillas, es fácil alquilar un vehículo que ya está afuera o a un cliente con la licencia vencida.

El objetivo del proyecto fue construir un sistema que registre la flota, los clientes y los contratos, y que aplique de forma automática las reglas de la agencia: precio según el tipo de vehículo, licencia exigida, edad mínima y estado de cada vehículo. Como segunda etapa se incorporó un kiosco de autoservicio, al estilo de los que se usan en cines o locales de comida rápida, para que el cliente pueda alquilar sin pasar por el mostrador.

La cátedra exige Java con interfaz gráfica, un diagrama de clases UML y una defensa con teoría, demostración en vivo y preguntas individuales. Quedaron fuera de alcance la tarifa semanal, la tarifa por kilómetro y el recargo por devolución tardía, y el sistema no tiene inicio de sesión ni roles.

Metodología

El trabajo se dividió en historias de usuario cortas, cada una con una sola tarea y escrita desde el punto de vista de quien usa el sistema (Cohn, 2004). Antes de programar cada historia se definieron dos o tres criterios de aceptación en formato *dado, cuando, entonces* y un plan breve.

Cada historia se programó con tests primero: se escribía una prueba que fallaba, el código mínimo para que pasara y después se ordenaba el código (Beck, 2003). Al terminar cada historia se volvían a correr todas las pruebas y se probaba la interfaz a mano.

Se plantearon 21 historias. Se completaron 18; las tres restantes (tarifa semanal, tarifa por kilómetro y recargo por atraso) se sacaron del alcance para llegar a la fecha de entrega. Las historias del kiosco (17 a 21) se agregaron en la segunda etapa del proyecto.

Diseño del sistema

Arquitectura en capas

El código está organizado en cinco paquetes con responsabilidades separadas. La interfaz (vista) solo muestra datos y captura lo que escribe el usuario: nunca valida reglas. Los servicios (servicio) validan los datos, aplican las reglas del negocio y lanzan excepciones propias. El modelo (modelo) contiene las entidades y la clase Agencia, que las agrupa. La persistencia (persistencia) guarda y carga la agencia completa en el archivo agencia.dat, y el paquete excepciones define los errores del dominio. Esta separación permite probar todas las reglas sin abrir ninguna ventana.

Conceptos de programación orientada a objetos

La Tabla 1 resume dónde se aplica cada concepto exigido por la cátedra.

Tabla 1

Conceptos de programación orientada a objetos aplicados en el sistema

Concepto	Dónde se aplica	Para qué sirve
Herencia	Auto, Moto y Camioneta heredan de Vehiculo, que es abstracta	Comparten patente, marca, modelo, kilómetros, precio base y estado
Polimorfismo	getFactorTarifa() y getLicenciaRequerida() se redefinen en cada subclase	La tabla y el kiosco calculan el precio sin preguntar el tipo de vehículo
Interfaz	Tarifa, implementada por TarifaDiaria (patrón Strategy; Gamma et al., 1994)	Una tarifa nueva sería otra clase, sin modificar Contrato
Composición	Cliente contiene su LicenciaConducir	La licencia se crea con el cliente y no existe sin él
Agregación	Agencia agrupa vehículos, clientes y contratos	Un único objeto que se guarda y se carga entero
Encapsulamiento	Atributos privados; los cambios pasan por métodos como cambiarEstado()	El estado de cada objeto solo cambia de forma válida
Excepciones propias	AlquilerException y sus hijas VehiculoNoDisponibleException y ConductorNoHabilitadoException	Al ser *checked*, el compilador obliga a la interfaz a manejarlas
Serialización	Agencia y todo lo que contiene implementan Serializable (Oracle, 2018)	Los datos se conservan entre ejecuciones

Reglas del negocio

El precio diario es el precio base multiplicado por un factor: 1,0 para autos (1,15 si son automáticos), 0,6 para motos y 1,4 para camionetas (1,6 con tracción 4x4). Las motos

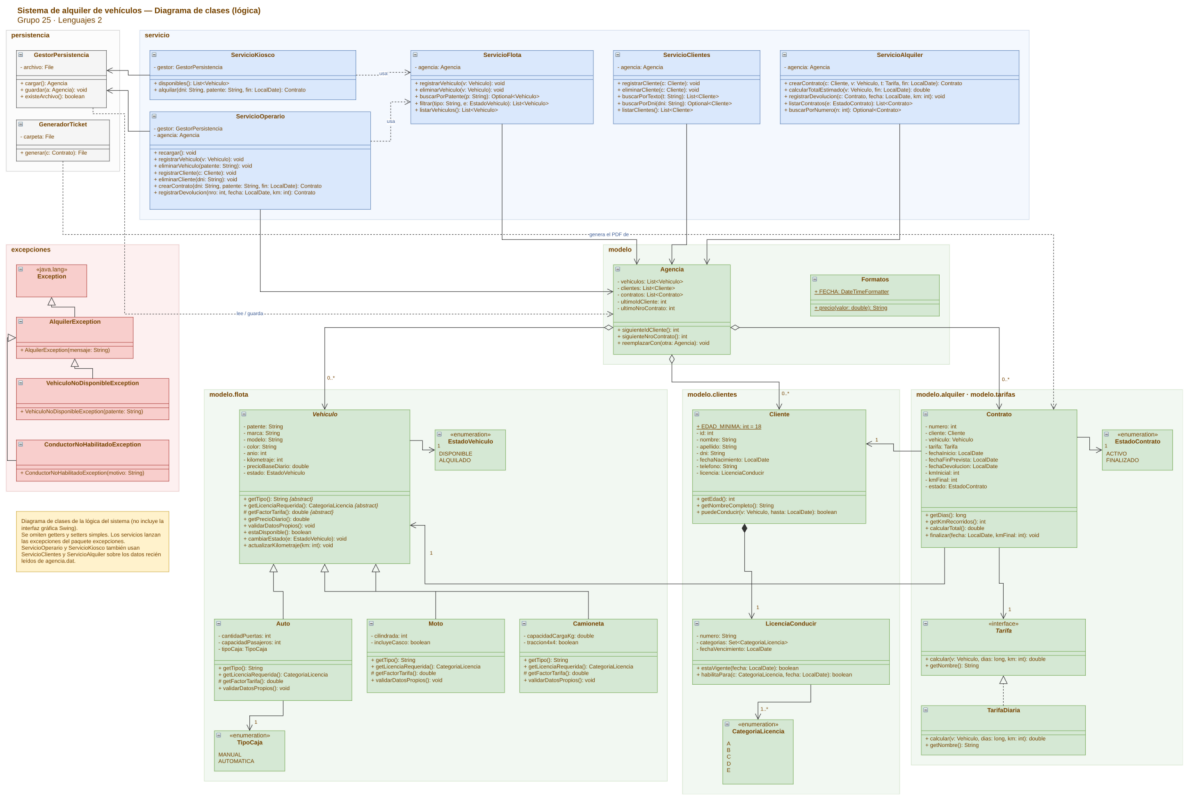
exigen licencia de categoría A, y los autos y camionetas, categoría B. Para alquilar, el cliente debe tener 18 años o más y la licencia vigente hasta la fecha de fin del contrato, y el vehículo debe estar disponible. En la devolución se cobra el mayor entre los días previstos y los reales.

Diagrama de clases

La Figura 1 muestra el diagrama de clases de la lógica del sistema. No incluye la interfaz gráfica ni los *getters* y *setters* simples. Sus atributos y métodos se tomaron directamente del código compilado, por lo que coinciden con la versión entregada.

Figura 1

Diagrama de clases de la lógica del sistema



Nota. La versión editable está en entrega/Diagrama_Clases_Logica_Grupo25.drawio y la versión para imprimir en entrega/Diagrama_Clases_Logica_Grupo25.pdf.

Funcionamiento simultáneo y ticket

Operario y kiosco sobre el mismo archivo

Las dos aplicaciones pueden estar abiertas al mismo tiempo y leen y escriben el mismo agencia.dat. El riesgo es que una pise lo que guardó la otra. Para evitarlo, cada cambio sigue tres pasos: vuelve a leer el archivo, aplica el cambio sobre esos datos recién leídos (ubicando el vehículo por patente, el cliente por DNI y el contrato por número) y guarda de forma atómica, escribiendo primero un archivo temporal y reemplazando después el original. Además, cada tres segundos las dos pantallas vuelven a leer el archivo: el operario ve los contratos hechos en el kiosco y el kiosco deja de mostrar los vehículos alquilados o dados de baja.

Kiosco de autoservicio

El kiosco muestra los vehículos disponibles como tarjetas grandes con la foto, la marca, el modelo y el precio por día. El cliente toca un vehículo, escribe su DNI y la fecha de devolución, ve el total y confirma. El alquiler pasa por las mismas validaciones que en el mostrador, y los errores se muestran en un mensaje grande sin guardar nada.

Ticket en PDF

Al confirmar, el kiosco genera tickets/contrato-N.pdf con el número de contrato, la fecha, el cliente y su DNI, el vehículo y su patente, la fecha de devolución, el precio por día y el total estimado, y lo abre en pantalla. El PDF se escribe con Java, sin bibliotecas externas, siguiendo la estructura del formato (International Organization for Standardization [ISO], 2008). Si el ticket no se puede generar, el alquiler igual queda registrado y el mensaje lo informa.

Verificación

El proyecto tiene 245 pruebas automáticas escritas con JUnit 5 (The JUnit Team, s. f.). Cubren el modelo, los servicios, la persistencia, el generador de tickets y parte de la

interfaz, como los modelos de las tablas y el formulario de clientes. Todas pasan sobre la versión entregada.

Además, se hizo una prueba automática de la interfaz gráfica: un programa abre el operario y el kiosco a la vez y recorre la demostración completa haciendo clic en los botones, completando formularios y esperando la recarga. Las 17 verificaciones de esa prueba dieron el resultado esperado, entre ellas que un contrato hecho en el kiosco aparece en el operario sin reiniciar, que un vehículo alquilado por el operario desaparece del kiosco y que los datos se conservan al cerrar y volver a abrir.

También se validó un ticket generado con un nombre con tildes, eñes y paréntesis: el archivo es un PDF válido y el texto se lee correctamente.

Organización de la defensa

La defensa dura 20 minutos: 7 de teoría, 8 de demostración en vivo y 5 de preguntas individuales. Cada integrante expone un tema de la teoría, de unos 70 segundos, y después muestra esa misma parte en la aplicación, en unos 80 segundos (Tabla 2). La presentación (entrega/Presentacion_Grupo25.pptx) sigue este orden y cada diapositiva tiene en sus notas qué decir.

Tabla 2

Reparto de la teoría y de la demostración entre los integrantes

Orden	Integrante	Tema de la teoría	Diapositivas	Paso de la demostración
1	Salazar, Alejandro Nicolás	Alcance y arquitectura	1 a 4	Abre el operario y el kiosco a la vez
2	Soto Llatser, José Ignacio	Herencia y polimorfismo en la flota	5	Registra una camioneta y aparece sola en el kiosco
3	Perez, Fernando F. Agustin	Clientes y licencias (composición)	6	Registra un cliente nuevo y lo busca
4	Juarez Ruiz, David Maximiliano	Contratos e interfaz Tarifa	7	Alquila un vehículo desde el operario
5	Chaparo, Samuel David	Validaciones y excepciones	8	Muestra un rechazo y un alquiler con ticket en el kiosco
6	Montoya, Tomás Benjamín	Kiosco, persistencia y ticket	9 y 10	Registra la devolución, cierra y reabre

Guion de la demostración

La demostración parte siempre de los datos de ejemplo de entrega/agencia-demo.dat, que se copian como agencia.dat antes de empezar (ver Apéndice A). Los pasos están encadenados, así que se hacen en este orden. Los resultados esperados de la Tabla 3 se comprobaron ejecutando esta misma secuencia con el código entregado sobre esos datos.

Tabla 3

Pasos de la demostración, datos que se cargan y resultado esperado

Paso	Integrante	Qué hace	Resultado esperado
1	Salazar	Abre ./ejecutar.sh y ./ejecutar-kiosco.sh	El kiosco muestra cuatro tarjetas: Fiat Cronos, Toyota Corolla, Honda CB190 y Toyota Hilux
2	Soto Llatser	Flota → Agregar camioneta: AE321JK, Ford, Ranger, Azul, 2024, 5000 km, precio base 50000, carga 900 kg, sin 4x4. Después intenta agregar un auto con patente AB123CD	La Ranger figura a $70.000 por día y aparece sola en el kiosco a los pocos segundos. La patente repetida se rechaza: «La patente AB123CD ya está registrada»
3	Perez	Clientes → Agregar cliente: Carla Núñez, DNI 45123456, nacida el 10/10/1995, licencia L-2001 de categoría B, vence el 10/10/2030. Después la busca por DNI	El cliente queda registrado y la búsqueda lo encuentra
4	Juarez Ruiz	Contratos → Nuevo contrato: Martín Díaz con la Toyota Hilux, devolución dentro de 3 días	Contrato 2 por $240.000; la Hilux pasa a ALQUILADO y desaparece del kiosco
5	Chaparo	En el kiosco toca el Corolla. Primero escribe el DNI 40222333 (Ana Torres); después el 38111222 (Lucía Gómez) con devolución dentro de 2 días	Ana es rechazada porque su licencia no tiene la categoría B. Lucía ve el total de $92.000, confirma, recibe el contrato 3 y se abre tickets/contrato-3.pdf
6	Montoya	En el operario muestra el contrato 3 y registra su devolución con la fecha de hoy y 8200 km. Después cierra el operario y lo vuelve a abrir	Se cobran 2 días (los previstos) y 200 km, total $92.000; el Corolla vuelve al kiosco. Al reabrir siguen todos los datos

Conclusiones

El sistema cumple las reglas de una agencia de alquiler y las aplica de la misma forma desde el mostrador y desde el kiosco, porque las dos interfaces usan los mismos servicios. Separar la interfaz de la lógica permitió probar todas las reglas sin abrir ventanas y agregar el kiosco sin cambiar el modelo.

Los conceptos de la cátedra tienen un uso concreto en el código: la herencia y el polimorfismo calculan el precio de cada tipo de vehículo, la interfaz Tarifa deja abierta la

incorporación de otras tarifas, la composición une a cada cliente con su licencia y las excepciones propias obligan a manejar cada rechazo.

Como mejoras quedan las tarifas semanal y por kilómetro, el recargo por atraso y el uso de una base de datos, que permitiría más de dos puestos trabajando a la vez sin depender de un archivo compartido.

Referencias

Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley.

Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

International Organization for Standardization. (2008). *Document management — Portable document format — Part 1: PDF 1.7* (ISO 32000-1:2008).

Oracle. (2018). *Java Platform, Standard Edition 11 API specification*. https://docs.oracle.com/en/java/javase/11/docs/api/

The JUnit Team. (s. f.). *JUnit 5 user guide*. https://junit.org/junit5/docs/current/user-guide/